

NODE.JS

COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam / interview favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

- ★ 1. Introduction to Node.js
- ★ 2. Modules (require / import)
- ★ 3. NPM (Node Package Manager)
- ★ 4. Creating a Basic HTTP Server
- ★ 5. Express.js Basics
 - 6. Routing in Express
- ★ 7. Middleware
 - 8. File System (fs module)
 - 9. Working with JSON
 - 10. Asynchronous Programming - Callbacks
- ★ 11. Promises and Async/Await
- ★ 12. Connecting to MongoDB
- ★ 13. REST API Basics
 - 14. Environment Variables (.env)

★ 1. Introduction to Node.js [IMPORTANT TOPIC]

Explanation:

Node.js lets us run JavaScript OUTSIDE the browser — directly on a computer or server. This means JavaScript, which was originally only for websites, can now also be used to build servers, APIs, and back-end applications.

Node.js is built on Google Chrome's V8 engine, which makes it very fast.

Key Points:

- Runs JavaScript on the server, not just in the browser.
- Great for building fast, scalable APIs and back-end systems.
- Comes with 'npm' (Node Package Manager) for installing extra tools and libraries.
- Uses non-blocking, asynchronous code — great for handling many users at once.

Example Code:

```
// app.js
console.log("Hello from Node.js!");
```

Output:

```
Hello from Node.js!
```

 *Note: Run a Node.js file from the terminal using the command: node app.js*

★ 2. Modules (require / import) [IMPORTANT TOPIC]

Explanation:

A module is a reusable piece of JavaScript code stored in its own file. Node.js has many BUILT-IN modules (like 'fs' for files), and we can also create our OWN modules to organize our code.

Example Code:

```
// math.js
function add(a, b) {
  return a + b;
}
module.exports = add;

// app.js
const add = require('./math');
console.log(add(5, 7));
```

Output:

12

💡 *Note: module.exports is used to make a function or value available to OTHER files using require().*

★ 3. NPM (Node Package Manager) *[IMPORTANT TOPIC]*

Explanation:

NPM is a huge online library of free, ready-made code packages that we can install and use in our Node.js projects, instead of writing everything from scratch.

Example Code:

```
# Create a new project (creates package.json)
npm init -y

# Install a package (example: express)
npm install express
```

Output:

```
+ express@4.18.2
added 57 packages
```

 *Note: package.json keeps track of every package your project depends on.*

★ 4. Creating a Basic HTTP Server *[IMPORTANT TOPIC]*

Explanation:

Node.js has a built-in 'http' module that lets us create a web server directly, WITHOUT needing any extra tools — this is what makes Node.js so useful for back-end development.

Example Code:

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.end('Hello from my server!');
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Output:

```
Server running on port 3000
```

★ 5. Express.js Basics [IMPORTANT TOPIC]

Explanation:

Express.js is the most popular framework built on top of Node.js, used to build web servers and APIs MUCH more easily than using the plain 'http' module directly.

Example Code:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Welcome to my Express app!');
});

app.listen(3000, () => {
  console.log('Server started on port 3000');
});
```

Output:

Server started on port 3000

6. Routing in Express

Explanation:

Routing means defining different URLs (routes) that respond with different content, like a 'Home' page and an 'About' page — a key part of building any website or API.

Example Code:

```
app.get('/', (req, res) => {  
  res.send('Home Page');  
});  
  
app.get('/about', (req, res) => {  
  res.send('About Page');  
});
```

Output:

```
Visiting '/' -> Home Page  
Visiting '/about' -> About Page
```

★ 7. Middleware *[IMPORTANT TOPIC]*

Explanation:

Middleware is a function that runs BETWEEN receiving a request and sending a response. It's used for tasks like logging, checking authentication, or reading data from a request, applied to every matching request automatically.

Example Code:

```
app.use((req, res, next) => {  
  console.log(`Request received: ${req.url}`);  
  next();    // move on to the next step  
});  
  
app.get('/', (req, res) => {  
  res.send('Home Page');  
});
```

Output:

```
Request received: /  
(Then the browser shows: Home Page)
```

 *Note: next() must be called inside middleware, or the request will get stuck and never reach its final destination.*

8. File System (fs module)

Explanation:

The built-in 'fs' (File System) module lets Node.js read from and write to files on the computer — useful for tasks like saving logs or reading configuration files.

Example Code:

```
const fs = require('fs');

// Writing to a file
fs.writeFileSync('notes.txt', 'Hello File!');

// Reading from a file
const content = fs.readFileSync('notes.txt', 'utf8');
console.log(content);
```

Output:

```
Hello File!
```

9. Working with JSON

Explanation:

JSON is the most common format for sending and receiving data in web APIs. Node.js can easily read incoming JSON and send JSON as a response.

Example Code:

```
app.use(express.json()); // lets Express understand JSON requests

app.post('/user', (req, res) => {
  console.log(req.body); // the JSON data sent by the client
  res.json({ message: 'User received!' });
});
```

Output:

(Sends back: { "message": "User received!" } as JSON)

10. Asynchronous Programming - Callbacks

Explanation:

Node.js is built around handling tasks that take time (like reading a file or a database call) WITHOUT freezing the whole program. A callback is a function passed into another function, to be run LATER once a task finishes.

Example Code:

```
const fs = require('fs');

fs.readFile('notes.txt', 'utf8', (err, data) => {
  if (err) {
    console.log('Error reading file');
    return;
  }
  console.log(data);
});

console.log('This runs FIRST, before the file is read!');
```

Output:

```
This runs FIRST, before the file is read!
Hello File! (printed a moment later)
```

★ 11. Promises and Async/Await [IMPORTANT TOPIC]

Explanation:

Promises and async/await are modern, cleaner ways to handle asynchronous tasks in Node.js, avoiding messy, deeply-nested callbacks (sometimes called 'callback hell').

Example Code:

```
const fs = require('fs').promises;

async function readMyFile() {
  try {
    const data = await fs.readFile('notes.txt', 'utf8');
    console.log(data);
  } catch (err) {
    console.log('Error reading file');
  }
}

readMyFile();
```

Output:

Hello File!

★ 12. Connecting to MongoDB [IMPORTANT TOPIC]

Explanation:

MongoDB is a popular NoSQL database that stores data as flexible, JSON-like documents. Node.js commonly connects to MongoDB using a library called 'Mongoose', to build the back-end for real apps.

Example Code:

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/mydatabase')
  .then(() => console.log('Connected to MongoDB'))
  .catch((err) => console.log('Connection error:', err));
```

Output:

```
Connected to MongoDB
```

★ 13. REST API Basics [IMPORTANT TOPIC]

Explanation:

A REST API is a standard way for a front-end (like a website or app) to communicate with a back-end server using simple HTTP methods: GET (read data), POST (create data), PUT (update data), and DELETE (remove data).

Example Code:

```
app.get('/users', (req, res) => { res.send('Get all users'); });  
app.post('/users', (req, res) => { res.send('Create a new user'); });  
app.put('/users/1', (req, res) => { res.send('Update user 1'); });  
app.delete('/users/1', (req, res) => { res.send('Delete user 1'); });
```

Output:

Each route responds based on the HTTP method and URL used to call it.

14. Environment Variables (.env)

Explanation:

Environment variables store sensitive or changeable settings (like database passwords or API keys) OUTSIDE the main code, usually in a special '.env' file, so they are never accidentally shared or exposed in the code itself.

Example Code:

```
// .env file
// DB_PASSWORD=mySecretPassword123

require('dotenv').config();
console.log(process.env.DB_PASSWORD);
```

Output:

```
mySecretPassword123
```

 *Note: The .env file should always be added to .gitignore, so it never gets accidentally uploaded to GitHub.*